



pdfapi

Техническое задание

Сервис, который превращает HTML, веб-страницы, Markdown и офисные документы в PDF по одному HTTP-запросу. Документ для всех, кто подключается к проекту: что мы строим, как это устроено и кто за что отвечает.

егор	бэкенд: аккаунты, ключи, квоты, оплата
команда	рендер: Gutenberg, конвертация, очередь
Claude	фронтенд: сайт, документация, кабинет

СОДЕРЖАНИЕ

- 1 Что мы делаем и на чём зарабатываем
- 2 Как проходит один запрос
- 3 Из чего состоит система
- 4 Публичный API — обещание клиентам
- 5 Тарифы, квоты и лимиты
- 6 Данные
- 7 Правила безопасности
- 8 Рендер — задание для команды
- 9 Бэкенд и фронтенд — что осталось
- 10 Решения, которые ещё не приняты
- 11 Как запустить у себя
- 12 Словарь

Редакция от 23 сентября 2026

1 Что мы делаем и на чём зарабатываем

Разработчикам постоянно нужны PDF: счета, акты, договоры, отчёты, выгрузки. Делать их самому значит держать в продакшене браузер без интерфейса (headless Chromium): он ест память, падает, теряет шрифты, ломается на кириллице. Мы забираем эту боль на себя.

Клиент отправляет нам HTML или ссылку — получает готовый PDF. Всё. Внутри мы используем **Gotenberg** — открытый сервер, который оборачивает Chromium и LibreOffice в удобный HTTP-интерфейс. Наша ценность не в самом рендере, а во всём вокруг него: ключи доступа, квоты, надёжность под нагрузкой, изоляция, понятные ошибки, документация.

Как зарабатываем

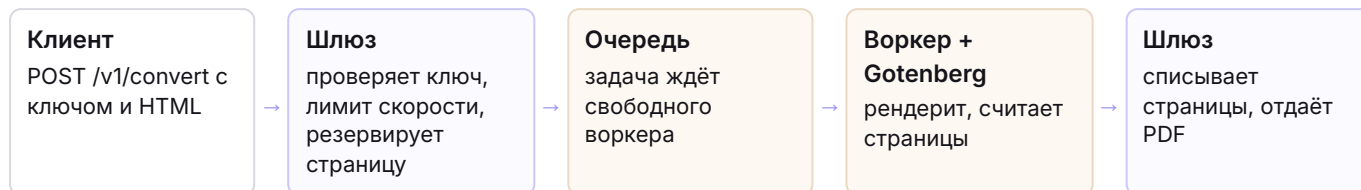
Платят за **страницы готового документа**, а не за запросы. Документ на сорок страниц стоит сорок страниц. Подписка помесечная, неизрасходованные страницы не переносятся.

Тариф	Цена	Страниц в месяц	Запросов в секунду	Что даёт
Free	0 ₽	100	1	только HTML, водяной знак, один ключ
Premium	1 990 ₽	10 000	10	все источники, без водяного знака, webhooks
Business	7 990 ₽	100 000	50	приоритетная очередь, SLA 99,9%

Цифры живут в двух местах — `web/src/lib/plans.ts` для сайта и `src/plans.ts` для сервера. Они обязаны совпадать: сайт показывает клиенту то, что сервер потом применяет.

2 Как проходит один запрос

Самое важное в документе. Если понятна эта схема — понятен весь проект.



□ бэкэнд — егор □ рендер — команда

- 1 **Шлюз узнаёт клиента.** Берёт ключ из заголовка, считает его хеш, ищет в базе (с кешем в Redis на минуту). Нет ключа — 401.
- 2 **Проверяет, можно ли.** Доступен ли этот источник на тарифе (URL, Markdown и Office — с Premium). Не превышена ли скорость запросов. Не кончилась ли месячная квота.
- 3 **Резервирует одну страницу.** Сколько страниц выйдет, заранее неизвестно — узнаем только по готовому файлу. Поэтому сначала «занимаем» одну, атомарно, чтобы два параллельных запроса не проскочили лимит.
- 4 **Отдаёт задачу рендеру.** Для ссылки — сначала проверяет, что она не ведёт во внутреннюю сеть.
- 5 **Рендер делает PDF** и возвращает файл плюс число страниц.
- 6 **Шлюз доначисляет остаток** (страниц было 7 — добавляет ещё 6). Если рендер упал — возвращает зарезервированную страницу.
- 7 **Отдаёт клиенту PDF** и заголовки с расходом, чтобы тот не делал второй запрос за остатком.

Граница ответственности

Рендер ничего не знает о клиентах, ключах и квотах. Он получает «что отрендерить» и возвращает «вот файл, в нём N страниц». Всё про деньги и доступ — в шлюзе. Это позволяет команде рендера работать полностью независимо.

3 Из чего состоит система

Часть	Технологии	Зачем	Порт
Сайт	Next.js 15, React 19, Tailwind 4	Лендинг, документация, тарифы, регистрация, кабинет с ключами, оплата	3000
Шлюз	Fastify 5, TypeScript	Весь HTTP: аккаунты, ключи, квоты, конвертация, оплата	3001
База	PostgreSQL	Пользователи, ключи, одноразовые токены, заказы	5432
Redis	ioredis	Сессии, кеш ключей, счётчики квот и лимитов, очередь	6379
Рендер	Gotenberg 8, BullMQ	Воркеры берут задачи из очереди и зовут Gotenberg	внутр.

Сайт и шлюз снаружи выглядят как одно целое: браузер ходит на `/api/*` того же адреса, а Next.js проксирует эти запросы на шлюз. Поэтому нет проблем с CORS и сессионная кука работает без ухищрений.

Два входа, которые нельзя смешивать

	Браузер человека	Программа клиента
Чем входит	сессионная кука <code>sid</code>	API-ключ <code>pdf_live_...</code>
Какие адреса	<code>/auth/*</code> , <code>/account/*</code>	<code>/v1/*</code>
Проверка в коде	<code>requireSession</code>	<code>authenticate</code>

Почему это важно

Если однажды объединить две проверки в одну «универсальную», сессионная кука браузера начнёт работать как полноценный API-ключ. Украли куку — получили доступ к конвертации за чужой счёт. Эти два хука никогда не сливаются.

Где что лежит

```
pdf/
src/
  config.ts db.ts redis.ts      — подключения и настройки
  auth.ts      — два хука: authenticate и requireSession
  key.ts session.ts tokens.ts   — ключи, сессии, одноразовые ссылки
  passwords.ts plans.ts quota.ts — пароли, тарифы, квоты
  routes/      — HTTP-обработчики
  render/      — NOB0E: зона команды рендера
migrations/    — SQL-схема базы
web/           — сайт целиком
```

4 Публичный API — обещание клиентам

Всё ниже уже опубликовано в документации на сайте. Это договор с клиентом: менять формат можно, но только вместе с документацией и осознанно.

Запрос

```
POST /v1/convert
Authorization: Bearer pdf_live_...      - или заголовок X-API-Key
Content-Type: application/json

{
  "source": "html",                      - html | url | markdown | office
  "html": "<h1>Счёт №42</h1>",           - или url / markdown / file
  "options": {                           - всё необязательно
    "landscape": false,
    "paperWidth": "8.27in", "paperHeight": "11.7in",
    "marginTop": "0.39in", "marginBottom": "0.39in",
    "marginLeft": "0.39in", "marginRight": "0.39in",
    "waitDelay": "0s"                   - подождать, пока скрипты дорисуют страницу
  }
}
```

Имена параметров совпадают с параметрами Gutenberg — это сделано намеренно, их можно передавать почти как есть.

Ответ

```
HTTP/1.1 200 OK
Content-Type: application/pdf
X-Pages-Rendered: 7
X-Quota-Limit: 10000
X-Quota-Used: 150
X-Quota-Remaining: 9850

<бинарный PDF>
```

Ошибки

Всегда JSON вида { "error": "код", "message": "текст" } .

Код	HTTP	Когда
missing_api_key	401	нет заголовка с ключом
invalid_api_key	401	ключ неизвестен или отозван
email_not_verified	403	владелец ключа не подтвердил почту
source_not_allowed	403	источник недоступен на тарифе
quota_exceeded	402	месячный лимит страниц исчерпан
rate_limit_exceeded	429	слишком часто; обязателен заголовок <code>Retry-After</code>
url_not_allowed	400	ссылка ведёт во внутреннюю сеть
render_failed	502	документ не отрендерился — обычно сломанная разметка
render_timeout	504	рендер не уложился в 30 секунд

5 Тарифы, квоты и лимиты

Квота — сколько страниц в месяц

Счётчик в Redis под ключом `quota:<пользователь>:<ГГГГ-ММ>`. Новый месяц — новый ключ, обнулять ничего не нужно, старый истекает сам.

- 1 **Резерв:** Lua-скрипт в Redis атомарно проверяет «хватит ли одной страницы» и прибавляет её.
- 2 **Рендер.**
- 3 **Списание:** `INCRBY` на (страниц – 1).
- 4 **Возврат при ошибке:** `DECRBY 1`.

Почему именно так, а не «прочитал, сравнил, прибавил»

Это проверено на практике в проекте: при ста параллельных запросах и лимите в сто страниц наивная схема «прочитать счётчик → сравнить → прибавить» пропустила **двести** запросов. Между чтением и записью успевают вклинуться соседи. Lua-скрипт выполняется в Redis целиком, без вклиниваний, — и пропускает ровно сто.

Следствие схемы: последний документ месяца может выйти за лимит — но не больше чем на себя самого. Следующий запрос уже получит 402. Это честнее, чем обрывать документ на середине.

Лимит скорости — сколько запросов в секунду

Алгоритм «ведро с жетонами»: у каждого ключа ведро на N жетонов (всплеск), оно пополняется со скоростью тарифа. Считается **по ключу**, а не по аккаунту — поэтому клиенту советуем отдельные ключи для продакшена и тестов: шумные тесты не отнимут скорость у прода.

Лимиты на вход и письма

Отдельно от API: вход ограничен пятью попытками в минуту на адрес и двадцатью на IP; восстановление пароля — тремя письмами за три часа. Два счётчика на входе потому, что каждый блокирует свою атаку: перебор паролей одного человека и перебор многих адресов с одной машины.

6 Данные

Таблица	Что хранит	Главное
<code>users</code>	почта, хеш пароля, дата подтверждения почты, тариф	пароль — только хеш <code>argon2id</code>
<code>api_keys</code>	владелец, название, хеш ключа, первые символы, даты создания, использования, отзыва	сам ключ не хранится нигде — только <code>SHA-256</code>
<code>email_tokens</code>	одноразовые ссылки из писем: подтверждение почты и сброс пароля	хеш, срок жизни, отметка об использовании
<code>orders</code>	попытки оплаты: кто, какой тариф, сумма, способ, статус	новая; создаётся при нажатии «оплатить»

В Redis лежит всё быстрое и временное: сессии (под хешем идентификатора), кеш найденных ключей на минуту, счётчики квот и лимитов, очередь рендера.

7 Правила безопасности

Каждое из этих правил появилось не из учебника, а после конкретной найденной дыры.

- **Секреты храним только хешем.** Ключи, сессии, ссылки из писем — SHA-256 (они длинные и случайные, подобрать нельзя). Пароли — argon2id (люди выбирают слабые пароли, нужен медленный хеш).
- **Одинаковый ответ для «неверный пароль» и «нет такой почты».** И одинаковое время ответа: для несуществующего адреса хеш всё равно считается. Иначе по разнице в миллисекундах можно проверить, кто у нас зарегистрирован.
- **Регистрация на занятую почту отвечает так же, как на свободную.** Настоящий владелец получает письмо «кто-то пытался».
- **В каждом запросе к базе — `user_id`.** Чужой ключ по его номеру не отозвать: получишь 404, как будто его нет, а не 403 «нельзя».
- **Смена пароля убивает все сессии** — иначе украденная сессия переживает смену пароля.
- **Кука сессии** — `httpOnly`, `SameSite=Lax`, в продакшене `Secure`.
- **Номер карты никогда не проходит через наш сервер.** Оплата на форме провайдера — иначе мы попадаем под требования PCI DSS.
- **Ссылки из запросов не ведут во внутреннюю сеть** — подробно в следующем разделе, это зона рендера.

8 Рендер — задание для команды

команда

Самостоятельный кусок: не зависит ни от кого и может начинаться сразу. Живёт в `src/render/`. Наружу отдаёт одну функцию — всё остальное внутри на ваше усмотрение.

Контракт: что вы отдаёте шлюзу

```
type PageOptions = {
  landscape?: boolean;
  paperWidth?: string; paperHeight?: string;
  marginTop?: string;   marginBottom?: string;
  marginLeft?: string;  marginRight?: string;
  waitDelay?: string;
};

type RenderInput =
  | { source: 'html';    html: string;    options?: PageOptions }
  | { source: 'url';     url: string;     options?: PageOptions }
  | { source: 'markdown'; markdown: string; options?: PageOptions }
  | { source: 'office';  file: Buffer; filename: string };

type RenderResult = { pdf: Buffer; pages: number; ms: number };

// Ошибки — только эти, шлюз превращает их в HTTP-коды:
// url_not_allowed → 400   render_failed → 502
// render_timeout  → 504   unavailable   → 503
class RenderError extends Error { code: 'url_not_allowed' | 'render_failed'
                                   | 'render_timeout' | 'unavailable' }

export function render(input: RenderInput): Promise<RenderResult>;
```

Шлюз (`routes/convert.ts`, пишет егор) вызывает только `render()`. Пока вы работаете, у него будет заглушка с тем же интерфейсом — и обе части можно делать одновременно.

Этапы

- 1 **Поднять Gutenberg** в Docker-контейнере. Важно: по умолчанию он слушает порт 3000 — тот же, что сайт в разработке. Пробросить на другой, например 3100.
- 2 **Синхронный `render()`** поверх Gutenberg: HTML, ссылка и Markdown — через Chromium, docx/xlsx/pptx — через LibreOffice. Параметры страницы передаются почти как есть.
- 3 **Подсчёт страниц** готового PDF — библиотекой вроде `pdf-lib`. На этом числе держится вся тарификация, ошибаться нельзя.
- 4 **Защита от чтения внутренней сети** — два уровня, оба обязательны:
 - ссылку из запроса резолвить самим и отклонять, если **хоть один** из её IP-адресов внутренний (127.x, 10.x, 172.16–31.x, 192.168.x, 169.254.x, ::1). Проверять после резолва, а не по тексту ссылки: домен может указывать на 127.0.0.1;
 - контейнер Gutenberg запускать **без маршрута во внутреннюю сеть**. Проверки входной ссылки мало: HTML может подтянуть картинку или стиль с внутреннего адреса, и Chromium пойдёт за ним сам.
- 5 **Таймауты**. Жёсткий потолок 30 секунд на документ. Бесконечный скрипт на странице не должен вешать воркер.
- 6 **Очередь и пул воркеров** на BullMQ поверх того же Redis. Chromium загружает процессор целиком: без очереди всплеск в тысячу документов уронит и шлюз, и базу. Business-тариф получает приоритет.
- 7 **Кириллица**. Прогнать документ на русском с разными начертаниями, убедиться, что нет «квадратиков». Если шрифтов в образе не хватает — доложить.

Критерии приёмки

Проверка	Ожидаем
HTML <code><h1>Привет</h1></code>	PDF на 1 страницу, текст читается, <code>pages = 1</code>
HTML на 40 страниц	<code>pages = 40</code> — ровно, не примерно
url = <code>http://169.254.169.254/</code>	<code>url_not_allowed</code> , Gotenberg даже не вызван
Домен, который резолвится в 127.0.0.1	<code>url_not_allowed</code>
HTML с картинкой <code>http://127.0.0.1:6379</code>	PDF собран, картинка не загружена
Страница с бесконечным циклом в JS	<code>render_timeout</code> через 30 с, воркер жив и берёт следующую задачу
50 одновременных задач	все выполнены по очереди, шлюз отвечает на другие запросы
docx-файл	PDF, число страниц посчитано
Gotenberg выключен	<code>unavailable</code> , а не зависание

9 Бэкенд и фронтенд — что осталось

Бэкенд егор

- 1 **Шесть исправлений из разбора кода.** Ограничитель частоты молча не работает — опечатка в имени команды Redis. «Выйти везде» ничего не удаляет — имена ключей не совпадают. `/auth/forgot` не отвечает на успешный запрос и висит. `resolveKey` обращается к несуществующей колонке. Заголовок альтернативного ключа назван `z-api-key` вместо `x-api-key`. Кука ставится без флага `Secure`.
- 2 **Квоты и лимит скорости по-настоящему** — сейчас `quota.ts` считает в памяти процесса и не переживает перезапуск.
- 3 **`routes/convert.ts`** — шлюз конвертации по схеме из раздела 2, с заглушкой `render()`, пока команда не закончит.
- 4 **Оплата:** таблица `orders`; `POST /account/checkout` создаёт заказ и возвращает ссылку на форму провайдера; `POST /billing/webhook` принимает подтверждение и меняет тариф; `GET /account/subscription` отдаёт состояние подписки.
- 5 **Идемпотентность:** повторный запрос с тем же ключом идемпотентности не списывает страницы второй раз. Вебхук провайдера тоже приходит повторно — второй раз тариф не продлевать.
- 6 **Тесты безопасности:** регистрация на чужую почту не меняет пароль; выход удаляет сессию; сброс убивает все сессии; в базе нет ни одного сырого ключа или пароля; квота выдерживает параллельные запросы.

Фронтенд Claude

Готово: лендинг, документация с конструктором запроса и поиском, тарифы с калькулятором, регистрация и вход, кабинет с ключами, страница оплаты, обратный отсчёт при 429, доступность в обеих темах.

- 1 **Реквизиты юрлица** на странице оплаты — ИНН и название при выборе оплаты по счёту.
- 2 **Онбординг в кабинете** — три шага до первого запроса с подставленным ключом. Включается, когда заработает конвертация.
- 3 **Сквозной прогон** от регистрации до отзыва ключа одной командой.
- 4 **Английская версия сайта.**

Где работа пересекается

- **Шлюз ↔ рендер:** только функция `render()` из раздела 8. Меняете сигнатуру — предупреждаете.
- **Шлюз ↔ сайт:** формат ответов `/auth/*`, `/account/*`, `/account/checkout`. Сайт уже ждёт ровно тот формат, что описан в плане оплаты.
- **Все ↔ клиенты:** раздел 4. Изменили API — поправили документацию в том же заходе.

10 Решения, которые ещё не приняты

Вопрос	От чего зависит
Домен	Превью ссылок, sitemap, адрес API в примерах документации. Пока везде localhost; меняется одной строкой в <code>.env</code> .
Платёжный провайдер	ЮKassa, CloudPayments или API банка — от выбора зависят формат вебхука и проверка подписи.
ИП или ООО	Без расчётного счёта оплата по счёту для компаний невозможна — останутся карта и СБП.
Где живёт рендер	На одной машине со шлюзом Chromium будет отнимать процессор у API и базы. Для продакшена — отдельно.
Продление тарифа	Разрешаем ли повторную оплату того же тарифа и за сколько дней до конца периода.

11 Как запустить у себя

```
# 1. База и Redis
brew services start postgresql@16
brew services start redis
createdb pdfapi
psql pdfapi -f migrations/001_init.sql
psql pdfapi -f migrations/002_auth.sql

# 2. Gotenberg — на 3100, потому что 3000 занят сайтом
docker run --rm -p 3100:3000 gotenberg/gotenberg:8

# 3. Шлюз — из корня проекта, порт 3001
npx tsx --watch --env-file=.env tests/serve.ts

# 4. Сайт — из папки web, порт 3000
cd web && npm run dev
```

Открывать `localhost:3000`. Письма никуда не уходят — ссылка подтверждения почты печатается в терминале шлюза.

В `.env` нужны как минимум `DATABASE_URL`, `REDIS_URL` и `APP_URL=http://localhost:3000`. Флаг пишется через дефис: `--env-file`, а не `--env_file` — неверное написание молча игнорируется.

12 Словарь

Шлюз	Наш сервер на Fastify. Всё, что приходит снаружи, проходит через него.
Рендер	Превращение HTML или документа в PDF. Делает Gotenberg по команде воркера.
Gotenberg	Готовый открытый сервер с Chromium и LibreOffice внутри. Принимает файл по HTTP, возвращает PDF.
Воркер	Отдельный процесс, который берёт задачи из очереди и выполняет их по одной.
Очередь	Список задач в Redis. Сглаживает всплески: тысяча запросов разом не превращается в тысячу браузеров разом.
Квота	Сколько страниц клиент может сделать за месяц по тарифу.
Лимит скорости	Сколько запросов в секунду разрешено одному ключу.

API-ключ	Длинная секретная строка <code>pdf_live_...</code> , по которой программа клиента доказывает, кто она.
Сессия	Запись о том, что человек вошёл на сайт. В браузере — кука, у нас — запись в Redis.
Хеш	Отпечаток строки, по которому нельзя восстановить саму строку. Храним отпечатки, а не секреты.
Вебхук	Запрос, который присылает нам чужой сервис, например платёжный провайдер: «деньги пришли».
SSRF	Атака, при которой нас заставляют сходить по ссылке в нашу же внутреннюю сеть и вернуть оттуда данные.
Идемпотентность	Свойство операции: повторить её два раза — то же самое, что один. Защищает от двойных списаний.